Traffic Sign Recognition with Voice Assistance Using Optimized Curvelet Entropy Features

Banhi Sanyal
Department of Computer Engineering
National Institute of Technology Kurukshetra
Kurukshetra, Haryana, India
banhi.sanyal@nitkkr.ac.in

Surja Sanyal
Department of Computer Science and Engineering
Indian Institute of Technology Guwahati
Guwahati, Assam, India
surja.sanyal@iitg.ac.in; surja.sanyal@iitbhu.ac.in

***Abstract*—Traffic Sign Recognition (TSR) is a vital component of Intelligent Transport Systems, aiming to reduce human intervention during driving. Current methods suffer from high computational complexity and require extensive training data. A two-stage framework for real-time TSR using optimized curvelet entropy descriptors is proposed to address these shortcomings. The first stage involves an efficient preprocessing module, followed by feature extraction using Fast Discrete Curvelet Transform variants and Shannon Entropy. The second stage converts the recognized traffic sign into speech using a text-to-speech engine, ensuring awareness regardless of the driver's level of concentration. The proposed framework achieves accuracies of 86.75%, 97.25%, and 98.88% on benchmark datasets, namely, the Indian Road Sign Database version 1.0 (IRSDBv1.0), the Belgian Traffic Sign Classification (BTSC), and the German Traffic Sign Recognition Benchmark (GTSRB), respectively.**

***Index Terms*—Voice assistance, Traffic sign recognition, Efficient preprocessing techniques, Texture features, Fast curvelet**

I. Introduction

Intelligent Transport System (ITS) aims to reduce human interference while driving. This approach has prompted the research community to further study Traffic Sign Recognition (TSR) systems. An investigation into scholarly work on TSR systems reveals that research incorporating the latest state-of-the-art techniques remains insufficient, despite the field's increased interest. It is observed that not all works are done on standard datasets. Furthermore, minimal work has been reported using an Indian dataset. The lack of an Indian dataset is also a rising concern, as India has a major number of road accidents. A work on the curvelet algorithm using the cuckoo search algorithm as a classifier was done by [1]. The study [2] used curvelet transform as a feature extractor. The study [2] used Locality Sensitive Discriminant Analysis (LSDA) to reduce dimensions, and Extreme Learning Machine (ELM) was deployed as a classifier. The research aimed to send warning signals from recognized traffic signs to the driver's Android device. Similar works were done in [3], [4], [5]. Using the curvelet descriptor is beneficial because it was specifically designed to efficiently represent objects with smooth curves, such as traffic signs. Curvelets can also reconstruct images even with missing data. Modules like curvelets offer an edge over classical feature extractors such as HAAR or Histogram of Oriented Gradients (HOG) variants. For example, the competition in the International Joint Conference on Neural Networks (IJCNN) [6] utilizes local descriptors from the GTSRB dataset, including HAAR and HOG variants. HOG 1 and HOG 2 have over 1500 features per image, whereas HOG 3 alone has nearly 2900 features. Since there are $39,209$ images, the training set's dimension with HOG 2 is almost $39,209 \times 1500$, which is enormous for any standalone device. Deep learning (DL) frameworks have certain limitations in capturing an automated set of descriptors. In addition, despite the efficiency of NNs as feature extractors, the handcrafted feature extraction module offers additional advantages and produces a shorter feature vector with simple computation. As the proposed method only has an efficient feature vector of length 42, FDCT with SE is preferred. The major aspects of this research are: 1) Feature dimension used for classification is optimized. 2) Suitable for real-time embedded systems due to low computational time. 3) Performance and recognition time are optimized. 4) Does not require additional hardware.

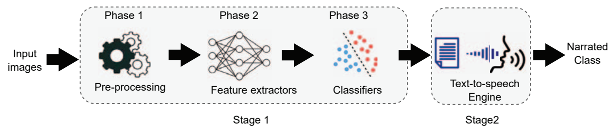
II. Proposed frameworks

The proposed methodology consists of two stages, as shown in Fig. 1(a). Stage 1 consists of the proposed TSR frameworks, and Stage 2 represents a text-to-speech engine that narrates the recognized class to the driver. The proposed frameworks comprise preprocessing, feature extraction, normalization, and classification modules. The two stages of the scheme are.

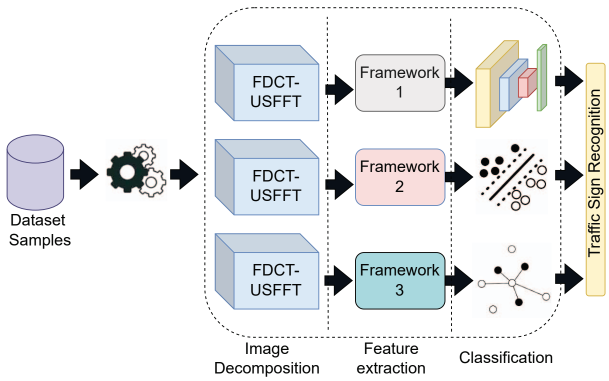
- Stage 1 of the framework is used for recognition and has the following steps. First, a dedicated preprocessing module is designed. Then, an efficient feature vector is extracted. For this, FDCT-USFFT and FDCT–WR are used for image decomposition and coefficient selection based on Shannon Entropy. This is followed by the use of FCNN, k-SVM, and kNN for recognition (Fig. 1(b)).
- Stage 2 of the framework is used for the text-to-speech engine and narrates the recognized class to the driver.

A. preprocessing

The primary focus of the preprocessing module is to develop a systematic approach that accounts for the characteristics of the TSR dataset and strikes a practical balance between time complexity and achieved efficiency. Two evaluation parameters



(a) A schematic diagram.



(b) The three phases of stage 1.

Fig. 1. Proposed methodology.

have been used to measure the noise in the input and preprocessed images. The Mean Squared Error (MSE) is the simplest measure of image quality. A lower MSE value is desired for higher image quality. A lower Peak Signal-to-Noise Ratio ($PSNR$) indicates poor image quality. The difference between an image and its preprocessed version is lower when the $PSNR$ value is modest. The proposed preprocessing module has the following steps:

1) Denoising aims to select the optimal smoothing operation and kernel size for TSR images through experimentation.
   a) In this step, 30 random images are taken; three different filters, namely mean, median, and adaptive median filters, are applied with the fixed kernel size of 5×5. For each filter, the average $PSNR$ and MSE are calculated and tabulated in Table I.
   b) Table I is inferred to observe the following trends. First, every filter is noted for its performance in terms of high PSNR and low MSE. The median filter gives lower PSNR and MSE values for IRSBv1.0 and GTSRB. Second, the performance should be consistent for all three datasets. Thirdly, the median filter performs poorly for IRSDBv1.0 or GTSRB (low PSNR and high MSE). Even for BTSC, only a slight difference is noted between PSNR and MSE values. The adaptive median, however, does not exhibit these limitations and has performed most efficiently in the dataset. Hence, adaptive median filtering is selected.
   c) The following experiment is conducted to determine the most suitable kernel size using 5×5, 7×7, and 9×9. Naturally, a 9×9 kernel gives the best results, as a larger kernel size ensures that any details that do not contribute to the median value are eliminated.

TABLE I
AVERAGE $PSNR$ AND MSE VALUES OF FILTERS

Dataset	**Applied filter**	$\boldsymbol{PSNR}$ **(in dB)**	$\boldsymbol{MSE}$
IRSDBv1.0	Mean	26.65	25.22
	Median	29.80	31.43
	Adaptive median	**34.56**	**30.62**
GTSRB	Mean	33.42	29.42
	Median	38.68	40.46
	Adaptive median	**45.56**	**29.78**
BTSC	Mean	34.12	27.31
	Median	29.02	28.13
	Adaptive median	**41.65**	**31.81**

   Thus, an adaptive median filter of size 9×9 is used in the first phase of denoising, removing unwanted information from the images.
2) The next step was morphological operations. Morphology is a broad set of image processing operations that process images based on shapes. Morphological operations apply a structuring element to an input image, creating an output image of the same size. In a morphological operation, the value of each pixel in the output image is based on comparing the corresponding pixel in the input image with its neighbors. Erosion followed by dilation was applied to the images. Hence opening operation was performed on the images. Opening is generally used to restore or recover the original image to the maximum possible extent. Closing is applied to smooth the contour of the distorted image and fill in the narrow breaks and long, thin gaps. Closing is also used to eliminate the small holes in the obtained image.
3) The last step was contrast adjustment, where the pixel values were adjusted using histogram equalization. This same process is repeated both for testing and training images. The steps are detailed in Table II, where they are also compared with other recent preprocessing approaches. The images are converted to 48×48 pixels. This reduces the complexity. Then, histogram equalization is applied to the V channel.

B. Feature extraction

The details of feature extraction are explained below.

1) Fast Discrete Curvelet Transform (FDCT): The form of curvelet transform, Fast Discrete Curvelet Transform (FDCT), was developed by [11] to address the everyday challenges of curvelet transform. FDCT has two variants: the wrapping-based FDCT (FDCT-WR) and the unequally spaced fast Fourier transform-based FDCT (FDCT-USFFT). The FDCT transforms faster, simpler, and less redundantly than its previous version. Both the FDCT variations are implemented in this study. In both the variations, the number of orientations, or angles (L), 2^{nd} level should be 8 at least and a multiple of 4 other than the first and last scale [11]. If x_r, x_c is the number of rows and columns in the image, then the level of decomposition is $s = \lceil \log(min(x_r, x_c)) - 3 \rceil$. The values

TABLE II
COMPARISON OF PREPROCESSING STEPS

Authors	**preprocessing step**	**Justification of each step**
Abdullah et al. [1]	Use of Gaussian filter	Denoising
	RGB,YCbCr, HSV color spaces	Contrast and illumination invariant
	Resize image	Variation in input image size
Berraho et al. [2]	Crop	Working only on Region of Interest
	Resize	Variation in input image size
Cao et al. [7]	RGB to HSV	Contrast and illumination invariant
Gudigar et al. [8]	RGB to grayscale	Grayscale is contrast and illumination invariant
	Resize using bilinear transformation	Variation in input image size
	Scaling pixel intensities	Intensity variation
	Local contrast normalization	Contrast variation
Saha et al. [9]	Histogram equalization	Contrast variation
Peng et al. [10]	Resize using bilinear interpolation	Variation in input image size
Proposed method	**RGB to grayscale**	**Grayscale is contrast and illumination invariant**
	Denoising using Adaptive median filtering with 9×9 filter	**Denoising, 9×9 works better**
	Morphological filtering using erosion and dilation	**Removes various image processing problems**
	Contrast adjustment by histogram equalization	**Contrast variation**

of s and L are experimentally set to 5 and 16, respectively. The number of features thus generated is quite huge. Since the number of curvelet descriptors generated is quite high, direct use of these components will lead to a high-dimensional feature vector. As such, Shannon entropy is applied to extract fewer, yet more efficient, coefficients. As the curvelet descriptor generates symmetric coefficients at angles β and $\beta + \pi$ [11]. In this study, the symmetric components are removed, and a feature vector is created. A total of 130 components in FDCT-USFFT and 80 using FDCT-WR. After removing the symmetric components at β and $\beta + \pi$, FDCT-WR gives 66 vector coefficients, and FDCT-USFFT gives 42 vector length.

2) Shannon Entropy & Feature normalization: Thus, the entropy is not calculated for all components from the decomposition, but rather for a selected subset. The feature vectors are then normalized within $[0, 1]$ using the min-max technique, as non-normalized features often skew the results. The classification then works on the normalized feature vectors.

C. Classifiers

Three classifiers were chosen for the current study based on the following merits and demerits. FCNN is known for its high accuracy in various domains. However, its high computational complexity makes it difficult to incorporate into a framework. There are several reasons for choosing kNN as a classifier. It has low computational complexity and is more suitable than other classifiers for limited sample datasets, such as IRSDBv1.0. SVM is robust to overfitting. However, training SVMs can be slow and computationally intensive, particularly with large datasets. Additionally, SVMs are sensitive to noise, making preprocessing necessary in the proposed framework.

Fully Connected Neural Network (FCNN): FCNN has been trained with mini-batch stochastic gradient descent using Nesterov's accelerated gradient, with a batch size of 32. Categorical cross-entropy was used with a momentum of 0.9, a learning rate of 0.01, epochs of 50, and a batch size of 32. Five consecutive dense layers were used to maintain the framework's time complexity at a low level. The total number of parameters was 4852, with no non-trainable parameters.

TABLE III
STRUCTURE OF TRAFFIC SIGN CLASSIFICATION DATASETS

Dataset	**Country**	**Train samples**	**Test samples**	**Classes**
GTSRB	Germany	39209	12630	43
IRSDBv1.0	India	1397	294	49
BTSC	Belgium	2978	458	62(training) 53(testing)

K-Nearest Neighbor(kNN): Its ease of application is that it includes tuning the only significant parameter of kNN, k. To select the value of k, the value of k varies from 2 to 6, as shown in Figs. 3(i), 3(c) and3(f) and its impact on accuracy. Note that the accuracy is highest for $K = 4$ across all datasets.

Kernel Support Vector Machine (k-SVM): k-SVM maps high-dimensional feature space into a linearly separable space. Two steps are followed to apply k-SVM. In the first step, the hyperparameters of k-SVM are tuned. The construction of the optimal hyperplane, where the aim is to minimize ω is the normal vector of the sample separation surface, ϵ is the relaxation parameter, l is the number of samples, and C is the penalty factor. When ϵ is constant, the larger C is, the greater the impact on the objective function and the higher the accuracy of training samples, but the possibility of overfitting is greater. Conversely, appropriately lowering C can result in some misclassifications and improve the classifier's generalization ability. With the change in C, the highest accuracy was achieved at $C = 60$. Hence, the value of C was set at 60. In the next step, the kernel function for the SVM is selected. Linear, Polynomial, Radial Basis Function (RBF), and Sigmoid kernel functions achieved an accuracy of 69.43%, 68.32%, 72.91%, and 71.31%, respectively. Hence, the best-performing kernel is the (RBF).

1) Audio narration: The system is integrated with a text-to-speech voice assistance engine to narrate the recognized traffic sign to the driver. The gTTS (Google Text-to-Speech) library is used for audio narrations. In this, the predicted traffic sign is simultaneously copied as text into the assistance engine's input. The language and tone of the speech can be tailored to the driver's needs. The input text is narrated to the driver.

D. Dataset and Evaluation parameters

All the experiments have been conducted on an Intel Core, $i3$-$4005U$ ($1.70GHz$), $4GB$ RAM system. Three datasets have been used, namely GTSRB [6], IRSDBv1.0 [12], and BTSC [13]. The datasets are split into training and testing sets and are listed in Table III. During the training phase, the training set was split at a $7:1$ ratio to the validation set, which was used for hyperparameter tuning across all three classifiers. The three datasets used are imbalanced, meaning the number of images in each class varies significantly. This is shown in Fig. 2. Oversampling is adopted to address this limitation. Oversampling is preferred over undersampling, as

TABLE IV
CLASSIFIERS ON ENTROPY-BASED CURVELET FEATURES

Dataset	Feature	Feature set	Classifier	Precision (in %)	Recall (in %)	F1 Score (in %)	Accuracy (in %)
IRSDBv1.0	**FDCT-WR** with SE	66	FCNN	84.13	80.30	82.17	83.26
			kNN	59.61	68.38	63.70	65.24
			k-SVM	62.68	66.22	54.40	60.91
IRSDBv1.0	FDCT-USFFT with SE	42	FCNN	89.62	89.62	89.62	86.75
			kNN	66.67	67.48	67.07	70.96
			k-SVM	52.00	61.74	56.45	57.30
GTSRB	FDCT-WR with SE	66	FCNN	93.71	86.14	89.77	89.24
			kNN	85.11	80.66	82.82	84.45
			k-SVM	79.83	66.02	72.28	74.49
GTSRB	FDCT-USFFT with SE	42	FCNN	98.94	98.94	98.94	98.88
			kNN	81.82	77.29	79.49	81.62
			k-SVM	76.03	64.92	70.04	72.91
BTSC	FDCT-WR with SE	66	FCNN	87.73	80.34	83.87	87.99
			kNN	74.85	71.91	73.35	79.69
			k-SVM	87.29	97.49	90.88	86.46
BTSC	FDCT-USFFT with SE	42	FCNN	97.80	97.31	97.12	97.25
			kNN	94.48	96.07	95.26	96.99
			k-SVM	94.54	97.19	95.84	96.72

the IRSDBv1.0 dataset has fewer images, and undersampling is not a suitable choice in this case. The default Synthetic Minority Over-sampling Technique (SMOTE) was adopted to oversample all classes to the number of examples in the majority class. The majority class and the number of images are available from Fig. 2. The F1-score has been included in the study because it is a more effective evaluation metric than accuracy for imbalanced datasets, as it considers both false positives and false negatives. The achieved performance is measured using the recall/sensitivity (Rc), precision (Pr), F1 Score ($F1$), and accuracy (Acc) performance metrics.

E. Results and Analysis

The evaluation parameters used in leading works like [14], [15] and [16] are used in this article as well.

Results on IRSDBv1.0: Table IV gives the results of various classifiers on both feature sets. Fig. 3(a) compares the three classifiers on two feature sets. It can be observed that FDCT-USFFT with SE performs better for all three classifiers. FCNN achieved the best performance across all feature sets, whereas k-SVM performance was lower than expected. Additionally, FDCT-USFFT offers higher accuracy with a shorter feature vector length. The results for FCNN are given in Fig. 3(b). One-vs-all experiments are performed for all classes to further investigate the effectiveness of k-SVM. The results of the first three classes are shown in Table V. It can be observed that the accuracy for One-Vs-All experiments is relatively high. The result of kNN differs based on the value of K. The highest value was achieved at $K = 4$, shown in Table IV.

Results on GTSRB: The results on GTSRB are given in Tables IV and V and Figs. 3(d), 3(e) and 3(f). The inference of the results is similar. FCNN achieved the best performance across all feature sets, whereas k-SVM performance was lower than expected. Also, FDCT-USFFT achieves higher accuracy with a shorter feature vector. The results for FCNN are given in Fig. 3(e). One-vs-all experiments are performed across all classes to evaluate k-SVM's effectiveness. The results of the first three classes are tabulated in Table V. It can be observed that the accuracy for One-Vs-All experiments is quite high. The result of kNN differs based on the value of K. The highest value was achieved at $K = 4$, tabulated in Table IV.

TABLE V
ONE-VERSUS-ALL CLASSIFICATION PERFORMANCE OF K-SVM FOR THREE CLASSES (IN %)

Datasets	Class	$\boldsymbol{P}$	$\boldsymbol{R}$	$\boldsymbol{F1}$	$\boldsymbol{A}$
IRSDB	0 vs All	96.20	98.63	97.40	97.23
	1 vs All	95.65	98.79	97.21	97.40
	2 vs All	96.66	96.22	96.44	96.33
GTSRB	0 vs All	98.98	98.98	98.98	98.98
	1 vs All	97.72	97.49	97.61	97.47
	2 vs All	97.01	97.93	97.47	97.31
BTSC	0 vs All	98.49	94.79	96.56	95.20
	1 vs All	98.36	91.72	94.92	93.01
	2 vs All	95.07	91.46	93.23	90.83

Results on BTSC: The results on BTSC are given in Tables IV, V and Figs. 3(g), 3(h) and 3(i). The findings are similar to those in IRSDBv1.0 and GTSRB.

Analysis: The inferences from the experiments are:

1) NN gives the best accuracy out of all classifiers. FCNN gives the best results by being efficient in 1-D vectors.
2) k-SVM falls short in the case of multiclass classification, but the results show that k-SVM performs better than FCNN regarding the One-Vs-All classification. This is because SVM is primarily used for linear classification. Using a kernel does not yield competitive results in multiclass recognition. Hence, it can be efficiently used in frameworks that recognize emergency traffic signs. Examples of emergency signs are *danger* and stopping *men at work*. These signs are mandatory; failing to comply may pose a threat to life or health. Thus, k-SVM is more suitable for models of sensitive sign classification.
3) A difference in results is evident for all classifiers as they perform lower on the IRSDBv1.0 dataset. The number of images is much lower in IRSDBv1.0 than in BTSC and GTSRB. Hence, classifiers have degraded results on it.
4) The accuracy and loss graphs for FCNN on the datasets are given in Figs. 3(b), 3(e) and 3(h). The training accuracy and test accuracy differ by only a tiny margin. The graphs show that the FCNN model is efficient and does not suffer from overfitting or underfitting.
5) The following results were recorded during the experiments: preprocessing time = $0.18s$, feature extraction time = $0.01s$, Classification time = $0.09s$, prediction time = $0.01s$, text-to-speech conversion time = $0.01s$.

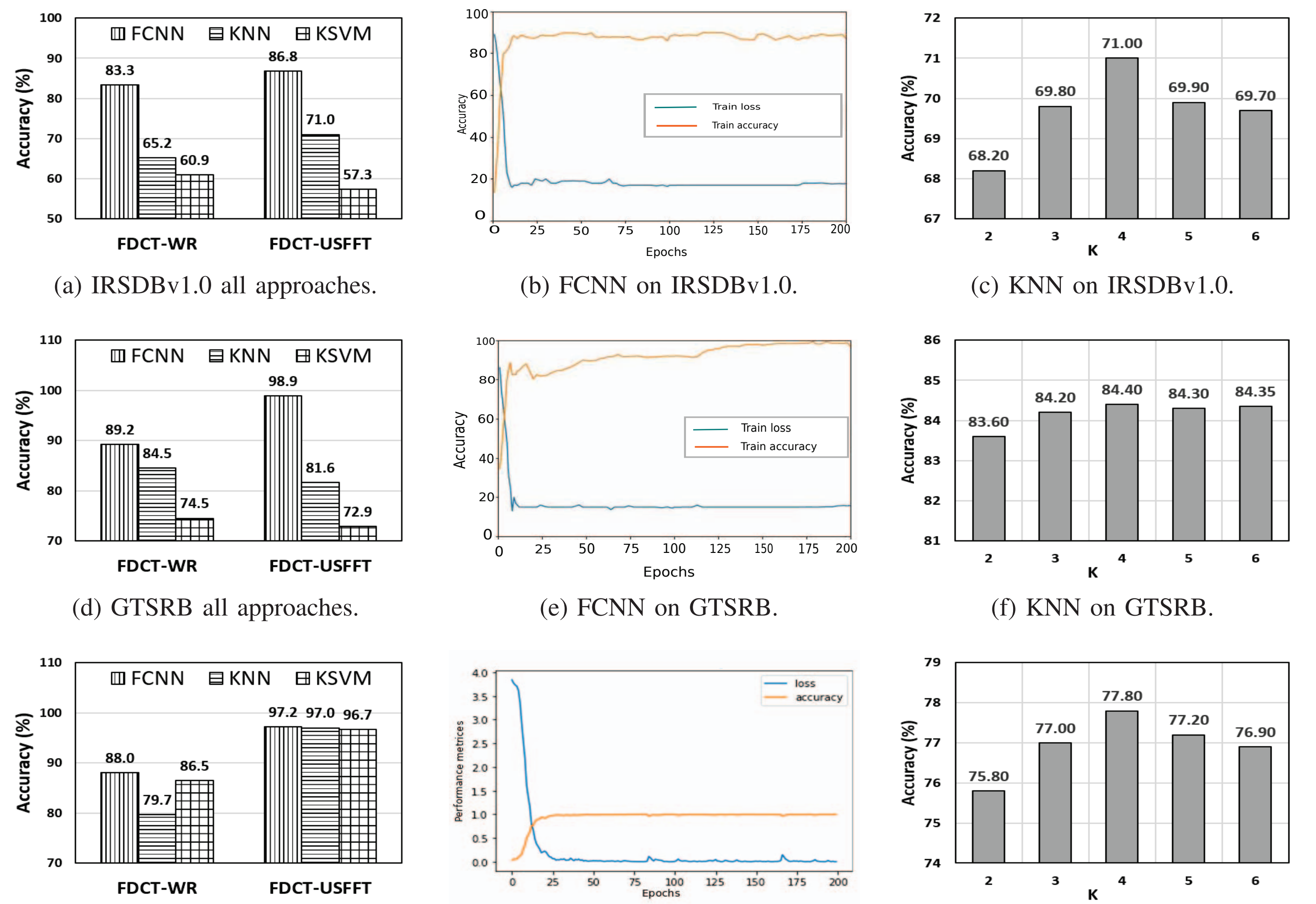
F. Comparative Results

The comparative results are presented in Table VI. The comparative literature study's inferences are as follows.

- It is essential to note that [1] has not considered 43 classes but only four categories for multiclass classification, which is why the accuracy is relatively high.
- Few studies record lower accuracies than the proposed.
- Studies like [2], [9], [19]- [22] use of NN or its variation to achieve high accuracy. The use of NN leads to:
  - Extra hardware requirements are recorded by [2], [7]–[9] as opposed to none for the proposed framework.

(a) (b) (c)

Fig. 2. Imbalance in various datasets. (a) GTSRB. (b) BTSC. (c) IRSDBv1.0.



(a) IRSDBv1.0 all approaches. (b) FCNN on IRSDBv1.0. (c) KNN on IRSDBv1.0.

(d) GTSRB all approaches. (e) FCNN on GTSRB. (f) KNN on GTSRB.

(g) BTSC all approaches. (h) FCNN on BTSC. (i) KNN on BTSC.

Fig. 3. Comparison of all approaches, FCNN training graph for FDCT-USFFT with SE approach, and effect of K on accuracy using three benchmark datasets.

– Higher parameters as listed in Table VI. Various studies did not focus on the number of parameters or report it.
– High computational complexity and hardware requirements, as shown in Table VII. Also, frameworks with higher feature dimensions per image will have longer recognition times than the proposed one.
– The major setback of using an NN is its degraded performance on small datasets, such as IRSDBv1.0, with fewer images in the training set. For example, [23] used a low-parameter (nearly 1.36 million parameters) Neural Network on IRSDBv1.0. Though the modified network, MDEffNet, performed well on GTSRB, it performed miserably on IRSDBv1.0. This is because IRSDB has fewer images than are required for a DL network. Thus, DL networks would fail on IRSDBv1.0.

TABLE VI
COMPARISON OF PERFORMANCE ON GTSRB

Authors	Feature	Classifier	Results (in %)	Parameters
Abdullah et al. [1]	Curvelet	Cuckoo research algorithm	Yellow=93 Green=94 Blue=94 Red=96	-
Berraho et al. [2]	Curvelet	LSDA+ELM	98.57	-
Cao et al. [7]	Segmentation	Variant LeNet-5	99.75	-
Gudigar et al. [8]	GIST	Graph based LDA with kNN	96.33	1024
Saha et al. [9]	-	Committee of CNN	99.92	13.72 M
Peng et al. [10]	-	Variant generalized autoencoder	90.47	1600
Mathias et al. [13]	I, PI, HOG	INNC	98.53	-
Jurišić et al. [17]	-	CNN	99.11	-
Wong et al. [18]	-	CNN	98.90	$5, 10, 000$
Bangquan et al. [19]	-	CNN	98.60	-
Ciregan et al. [20]	-	CNN	98.52	90 M
Huang et al. [21]	HOG	K-ELM	99.56	-
Mannan et al. [22]	-	CNN	99.00	-
Proposed method	**FDCT-USFFT +SE**	**FCNN**	**R=98.94 P=98.94 F1=98.94 A=98.88**	-

TABLE VII
COMPARISON OF SYSTEM REQUIREMENTS ON GTSRB

Authors	CPU	RAM
Berraho et al. [2]	$i5$, quadraple cores	$8GB$
Cao et al. [7]	Intel Core $i5 - 6500$ $(3.20GHz)$	-
Gudigar et al. [8]	Core $i5 - 4210U$ $(2.4GHz)$	$4GB$
Saha et al. [9]	NVIDIA GeForce $940MX$	-
Proposed method	**Intel Core $\boldsymbol{i3 - 4005U(1.70\ GHz)}$**	$\boldsymbol{4GB}$

III. CONCLUSION

TSR is a multiclass complex problem with high feature dimensions. Higher feature dimensions pose domain-specific challenges. A curvelet decomposition of traffic signs is adopted to address these challenges. Curvelet entropy features were used in this work for classification using three classifiers: FCNN, kNN, and k-SVM. The proposed method, utilizing FDCT-USFFT with SE and FCNN as a classifier, achieved accuracies of 86.75%, 97.25%, and 98.88% on IRSDBv1.0, BTSC, and GTSRB, respectively. The proposed scheme achieved optimized results with reduced feature vector length. Additionally, the classification technique uses only 42 features, significantly less than the other discussed methods.

SMOTE-NC may be used to evaluate its impact in the process. However, the scheme may exhibit degraded performance in the presence of occlusion, blur, broken or misaligned traffic signs, and multiple traffic signs in the same frame. In the future, the model's overall performance will be improved and customized with more datasets from various countries.

REFERENCES

[1] A. M. A. saadi Abdullah, A. and A. N. Ismael, "Traffic signs recognition using cuckoo search algorithm and curvelet transform with image processing methods," *Journal of AL-Qadisiyah for computer science and mathematics*, vol. 11, no. 2, pp. 74–81, 2019.

[2] S. Berraho, S. El Margae, M. A. Kerroum, and Y. Fakhri, "Texture classification based on curvelet transform and extreme learning machine with reduced feature set," *Multimedia Tools and Applications*, vol. 76, no. 18, pp. 18 425–18 448, 2017.

[3] K. S. N. Yadav, V. Aarthi, K. R. G. Sreenath, K. Kishan, M. Kumar *et al.*, "Deep learning based voice assisted traffic sign detection using rcnn," in *2024 International Conference on Recent Innovation in Smart and Sustainable Technology (ICRISST)*. IEEE, 2024, pp. 1–5.

[4] K. Zaman, S. Zhaoyun, B. Shah, T. Hussain, S. M. Shah, F. Ali, and U. S. Khan, "A novel driver emotion recognition system based on deep ensemble classification," *Complex & Intelligent Systems*, vol. 9, no. 6, pp. 6927–6952, 2023.

[5] K. Zaman, G. Zengkang, S. Zhaoyun, S. M. Shah, W. Riaz, J. C. Ji, T. Hussain, and R. W. Attar, "A novel emotion recognition system for human–robot interaction (hri) using deep ensemble classification," *International Journal of Intelligent Systems*, vol. 2025, no. 1, p. 6611276, 2025.

[6] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," *Neural networks*, vol. 32, pp. 323–332, 2012.

[7] J. Cao, C. Song, S. Peng, F. Xiao, and S. Song, "Improved traffic sign detection and recognition algorithm for intelligent vehicles," *Sensors*, vol. 19, no. 18, p. 4021, 2019.

[8] A. Gudigar, S. Chokkadi, U. Raghavendra, and U. R. Acharya, "An efficient traffic sign recognition based on graph embedding features," *Neural Computing and Applications*, vol. 31, no. 2, pp. 395–407, 2019.

[9] S. Saha, M. S. Islam, M. A. B. Khaled, and S. Tairin, "An efficient traffic sign recognition approach using a novel deep neural network selection architecture," in *Emerging Technologies in Data Mining and Information Security*. Singapore: Springer, 2019, pp. 849–862.

[10] X. Peng, Y. Li, X. Wei, J. Luo, and Y. L. Murphey, "Traffic sign recognition with transfer learning," in *2017 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2017, pp. 1–7.

[11] E. Candès, L. Demanet, D. Donoho, and L. Ying, "Fast discrete curvelet transforms," *Multiscale Modeling & Simulation*, vol. 5, no. 3, pp. 861–899, 2006.

[12] B. Sanyal, R. K. Mohapatra, and R. Dash, "Fully annotated indian traffic signs database for recognition," in *Soft Computing: Theories and Applications*. Singapore: Springer, 2020, pp. 695–703.

[13] M. Mathias, R. Timofte, R. Benenson, and L. Van Gool, "Traffic sign recognition-how far are we from the solution?" in *The International Joint Conference on Neural networks (IJCNN)* . IEEE, August 2013, pp. 1–8.

[14] M. Yang and S. Han, "Ets-yolo: An efficient yolo-based model for real-time traffic sign recognition," *Signal, Image and Video Processing*, vol. 19, no. 8, p. 650, 2025.

[15] X. Zhang, W. Wang, Q. Wang, L. Shan, S. Yamane, B. Zheng, S. Liu, T. Shen, and D. Ge, "Enhancing small traffic sign recognition based on an improved yolov8 algorithm," *Signal, Image and Video Processing*, vol. 19, no. 6, p. 438, 2025.

[16] A. J. Prakash and S. Sruthy, "Enhancing traffic sign recognition (tsr) by classifying deep learning models to promote road safety," *Signal, Image and Video Processing*, vol. 18, no. 5, pp. 4713–4729, 2024.

[17] F. Jurišić, I. Filković, and Z. Kalafatić, "Multiple-data set traffic sign classification with onecnn," in *3rd IAPR Asian Conference on Pattern Recognition (ACPR)*. IEEE, November 2015, pp. 614–618.

[18] A. Wong, M. J. Shafiee, and M. S. M. Jules, "A highly compact deep convolutional neural network architecture for real-time embedded traffic sign classification," *IEEE Access*, vol. 6, pp. 59 803–59 810, 2018.

[19] X. Bangquan and W. X. Xiong, "Real-time embedded traffic sign recognition using efficient convolutional neural network," *IEEE Access*, vol. 7, pp. 53 330–46, 4 2019.

[20] D. Ciregan, U. Meier, and J. Schmidhuber, "Multi-column deep neural networks for image classification," in *IEEE conference on computer vision and pattern recognition 2012 Jun 16*. IEEE, 2012, pp. 3642–3649.

[21] Z. Huang, . Y. Yu, J. Gu, and H. Liu, "An efficient method for traffic sign recognition based on extreme learning machine," *IEEE transactions on cybernetics*, vol. 47, no. 4, pp. 920–933, 2016.

[22] A. Mannan, K. Javed, K. S. Noon, and H. A. Babri, "Optimized segmentation and multiscale emphasized feature extraction for traffic sign detection and recognition," *J. Intelligent Fuzzy Sys.*, vol. 36, no. 1, pp. 173–188, 2019.

[23] B. Sanyal, R. K. Mohapatra, and R. Dash, "Real-time embedded tsr using mdeffnet: An efficient compact dnn on varied multi-resolution images across multiple databases," *International Journal of Wavelets, Multiresolution and Information Processing*, vol. 19, no. 03, p. 2050082, 2021.